Having thought about all these scenarios and more, you then need to consider additional factors about the test environment:

- What you mean by “multiple threads” in each case (3, 4, 1024?)
- Whether there are enough processing cores in the system for each thread to run on its own core
- Which processor architectures the tests should be run on
- How you ensure suitable scheduling for the “while” parts of your tests

There are additional factors to think about specific to your particular situation. Of these four environmental considerations, the first and last affect the structure of the test itself (and are covered in section 10.2.5), whereas the other two are related to the physical test system being used. The number of threads to use relates to the particular code being tested, but there are various ways of structuring tests to obtain suitable scheduling. Before we look at these techniques, let’s look at how you can design your application code to be easier to test.

10.2.3. Designing for testability

Testing multithreaded code is difficult, so you want to do what you can to make it easier. One of the most important things you can do is *design* the code for testability. A lot has been written about designing single-threaded code for testability, and much of the advice still applies. In general, code is easier to test if the following factors apply:

- The responsibilities of each function and class are clear.
- The functions are short and to the point.
- Your tests can take complete control of the environment surrounding the code being tested.
- The code that performs the particular operation being tested is close together rather than spread throughout the system.
- You thought about how to test the code before you wrote it.

All of these are still true for multithreaded code. In fact, I’d argue that it’s even more important to pay attention to the testability of multithreaded code than for single-threaded code, because it’s inherently that much harder to